DATABASE MANAGEMENT SYSTEMS • STUDY GUIDE

Modeling Techniques

Chapter 2 | Data Models & Entity Relationship Diagrams

2.1 Different Types of Models & Introduction to Entity Relationship Diagram (ERD)

Prepared for **OAVS Computer Teacher (TGT)**, **KVS**, **NVS**, **DSSSB**, **OSSSC**, **B.Ed Entrance**, **MCA**, **BCA**, **B.Tech** and University Examinations

A concept-first, exam-oriented reference · Diagrams · Tables · Solved Examples · Revision Notes

CHAPTER 2

Modeling Techniques

Before a database is built, its structure must be designed. This chapter explains how database designers think about data in stages — from abstract models down to Entity Relationship Diagrams that map real-world information into a blueprint a database can be built from.

What are Modeling Techniques?

Modeling techniques are the methods and notations used to represent the data of a system in an organised, visual form before it is actually built. Rather than jumping straight into creating tables, a designer first models *what* data exists and *how* it is related — using data models and diagrams such as the ER Diagram. This makes the design easier to understand, verify, and translate into a working database.

Syllabus Coverage — 2.1

- Different Types of Models (Conceptual, Logical, Physical)
- Introduction to Entity Relationship Diagram (ERD)

Table of Contents

2.1 Different Types of Models 4

Concept and Need for Data Models 4

Characteristics of a Good Data Model 4

Conceptual Data Model 6

Logical Data Model 6

Physical Data Model 7

Comparison of Conceptual, Logical & Physical Models 9

Revision Aids — Section 2.1 10

2.2 Introduction to Entity Relationship Diagram (ERD) 11

Definition, Purpose and Need for ERD 11

Entity and Entity Set 12

Attributes and Types of Attributes 12

Relationship and Degree of Relationship 14

Cardinality and Participation Constraints 15

Keys used in ERD 17

Naming Conventions & Steps to Draw an ER Diagram 18

Advantages, Limitations and Applications of ERD 19

Common Errors while Drawing ERDs 20

Worked Examples (Student, Library, Hospital, Banking) 21

Revision Section 26

One-Page Quick Revision Notes 26

32 One-Line Revision Points 27

Expected Viva Questions 28

2.1 Different Types of Models

2.1.1 Concept and Need for Data Models

Definition: A **data model** is a conceptual tool — a collection of concepts, notations and rules — used to describe the structure of a database: what data it holds, how pieces of data relate to one another, and what constraints apply to that data. In simple words, a data model is a blueprint that shows *what* data will be stored and *how* it will be organised, before a single table is actually created.

Introduction: Just as an architect draws a plan before a building is constructed, a database designer builds a data model before a database is implemented. The data model acts as a communication bridge between the people who understand the business requirements (users, analysts) and the people who build the system (programmers, DBAs).

> **EXAM POINT**
>
> A data model = Structure (entities/tables) + Relationships + Constraints + Operations that can be performed on the data.

Purpose / Need for Data Models

- To represent data in an understandable, organised form before implementation.
- To ensure that all stakeholders (users, designers, developers) have the same understanding of the data.
- To identify entities, their attributes and the relationships between them clearly.
- To reduce redundancy and ambiguity in the database design at an early stage.
- To act as a bridge between real-world requirements and the physical database.
- To simplify communication and act as documentation for future maintenance.

Characteristics of a Good Data Model

- **Simplicity:** Easy to understand for both technical and non-technical people.
- **Clarity/Completeness:** Captures all the data requirements of the organisation without leaving gaps.
- **Non-redundancy:** Avoids repeating the same fact in more than one place.
- **Accuracy:** Correctly reflects real-world business rules and constraints.
- **Sharability:** Can be understood and reused by different teams/applications.
- **Flexibility:** Can accommodate future changes with minimum rework.
- **Correspondence with reality:** Every element of the model should map to a genuine real-world object or fact.

> **REMEMBER**
>
> A good data model is judged by how well it balances *simplicity* for humans with *precision* for machines.

Types of Data Models Covered in this Syllabus

Data models are generally studied in three stages that move from an abstract, human-friendly view to a machine-specific view: **Conceptual**, **Logical**, and **Physical**.

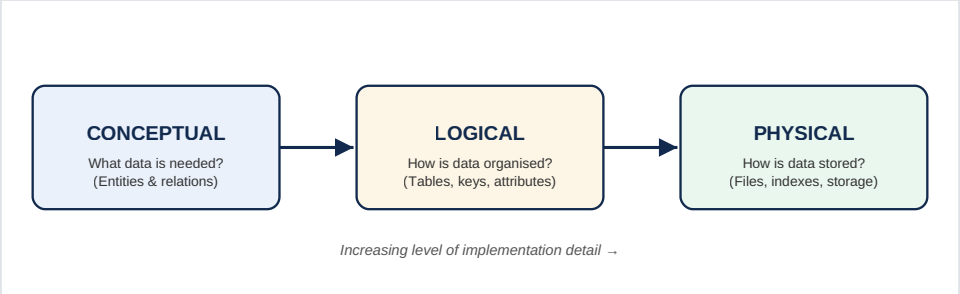


Fig. 2.1 — Progression of Data Models: Conceptual → Logical → Physical

Design moves from a broad, technology-independent view (Conceptual) to a structured, database-independent view (Logical) and finally to a technology-specific storage view (Physical).

2.1.2 Conceptual Data Model

Definition: The conceptual data model gives a high-level, technology-independent view of the whole database. It focuses purely on *what* data exists and how the major data objects relate to each other — not on how they will be stored.

Features / Characteristics

- Represents entities and relationships only, in a simple, non-technical form.
- Independent of any DBMS software or hardware.
- Created early, usually as the first output of requirement analysis.
- Understandable by business users, not only technical staff.
- Typically drawn as an Entity Relationship Diagram (ERD).

Advantages

- Easy for non-technical stakeholders to validate requirements.
- Provides a strong foundation before deeper design decisions are made.
- Helps catch missing or wrong requirements early, when they are cheapest to fix.

Disadvantages

- Too abstract to be implemented directly.
- Does not specify data types, keys or storage details.

Real-life example: A simple sketch showing that a "Student" is enrolled in a "Course", without worrying about column names, data types, or which database software will be used.

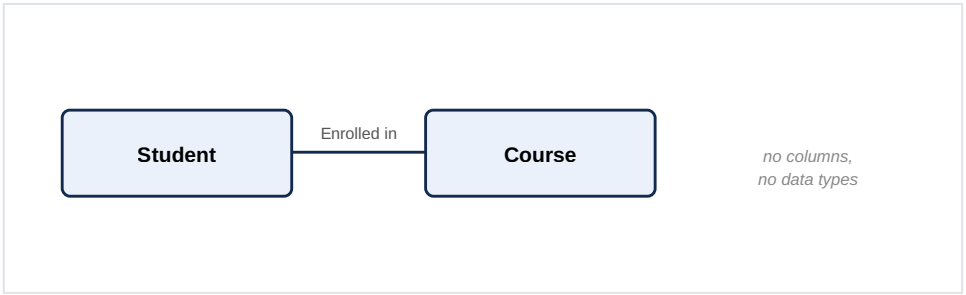


Fig. 2.1(a) — Conceptual Model example: only the entities and their relationship are shown

Nothing about columns, keys or storage is decided yet — the goal at this stage is simply to agree on *what* the two things are and *that* they are related.

2.1.3 Logical Data Model

Definition: The logical data model adds structural detail to the conceptual model. It defines entities as tables, attributes as columns, and formally establishes primary and foreign keys and relationships — but is still independent of any specific DBMS product.

Features / Characteristics

- Data is organised into entities/tables with defined attributes.
- Primary keys, foreign keys and relationships are explicitly identified.
- Independent of a specific DBMS (works whether the target is MySQL, Oracle, or SQL Server).
- Normalisation rules are usually applied at this stage (in advanced modelling — kept only as a reference point here).

Advantages

- Bridges the gap between abstract business requirements and physical implementation.
- Ensures consistent structure regardless of which database product is finally used.

Disadvantages

- Requires more technical knowledge than a conceptual model.
- Still cannot be executed directly on hardware without a physical model.

Real-life example: Defining a "STUDENT" table with columns Student_ID (Primary Key), Name, Date_of_Birth, and a "COURSE" table with Course_ID (Primary Key), linked through a foreign key.

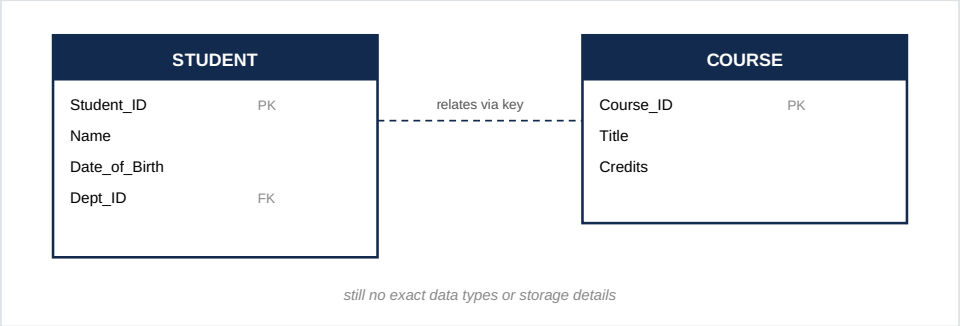


Fig. 2.1(b) — Logical Model example: tables, columns, and Primary/Foreign keys are defined

The entities become tables with named columns and identified keys — enough to design the schema — but the exact data types and physical storage are still undecided.

2.1.4 Physical Data Model

Definition: The physical data model describes exactly how data will be stored on physical storage media using a specific DBMS. It is the most detailed model and is used directly to build the database.

Features / Characteristics

- Specifies exact data types, field lengths, indexes and storage parameters.
- Dependent on the specific DBMS chosen (e.g., MySQL, Oracle, PostgreSQL).
- Includes details such as file organisation, indexing strategy and partitioning.
- Prepared by database administrators (DBAs) closest to implementation time.

Advantages

- Directly usable to create the actual database.

• Optimised for performance, storage efficiency and access speed.

Disadvantages

• Highly technical — difficult for non-technical stakeholders to review.

• Tied to a specific DBMS; changing the DBMS may require redesign.

Real-life example: Creating the actual table with `Student_ID INT PRIMARY KEY, Name VARCHAR(50), DOB DATE` inside a specific DBMS, along with an index on Student_ID for faster search.

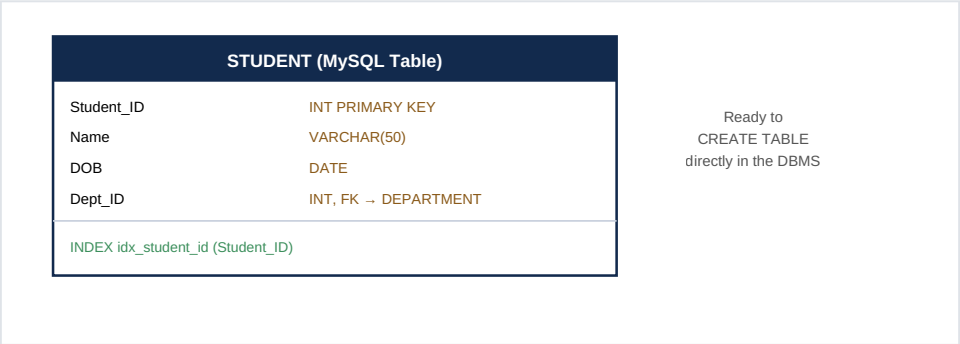


Fig. 2.1(c) — Physical Model example: exact data types, keys and an index, ready for implementation

This is the version an administrator can hand straight to the database as a `CREATE TABLE` statement — every detail needed for storage and performance is now specified.

REMEMBER

Placed side by side, Fig. 2.1(a)–(c) show the same STUDENT–COURSE idea growing more detailed at each stage: entities only → tables with keys → exact columns, types and indexes.

SHORTCUT TO REMEMBER THE ORDER

"C-L-P": **C**onceptual (What) → **L**ogical (How organised) → **P**hysical (How stored). Think: *Idea → Design → Build.*

2.1.5 Comparison of Conceptual, Logical and Physical Models

Table 2.1 — Conceptual vs Logical vs Physical Data Model

Basis	Conceptual Model	Logical Model	Physical Model
Level of detail	Very high level / abstract	Moderate — structural detail	Very detailed / implementation level
DBMS dependency	Independent	Independent	Dependent on specific DBMS
Focus	What data is needed	How data is organised (tables, keys)	How data is physically stored
Audience	Business users, analysts	Designers, developers	Database administrators
Typical output	ER Diagram (basic)	Detailed ER/relational schema	Actual database tables & indexes
Contains data types?	No	Rarely / generic	Yes, exact data types
Stage of design	First stage	Second stage	Final stage

Practical Use Cases of Each Model Type

Table 2.1(a) — Where Each Model is Used in Practice

Model	Typically prepared by	Typical deliverable / tool	Practical use case
Conceptual	Business analyst, requirement team	Whiteboard sketch, basic ER diagram	Client meetings to confirm "what data matters" before coding starts
Logical	System / database designer	Detailed ER diagram with keys, normalised schema	Design review documents shared with the development team
Physical	Database Administrator (DBA)	DDL scripts (CREATE TABLE), index plans	Actually creating and tuning the production database

Important Points to Remember

- A single conceptual model can be translated into several different logical models depending on design choices.
- A single logical model can be implemented as different physical models on different DBMS products (e.g., the same logical design implemented in MySQL versus Oracle).
- Moving from Conceptual → Logical → Physical adds detail; it never removes information gathered earlier.
- Changes discovered late (at the physical stage) are the most expensive to fix — this is why careful conceptual modelling matters.

> **EXAM TIP**
>
> If a question asks to "arrange in order" or "identify the first step," the safe answer sequence is always **Conceptual → Logical → Physical**. If asked which model is closest to the end-user's understanding, the answer is **Conceptual**; if asked which is closest to the machine, the answer is **Physical**.

Chapter Summary — Section 2.1

A data model is a blueprint describing what data will be stored, how it is organised, and what rules govern it. A good data model is simple, complete, non-redundant, accurate and flexible. Design typically proceeds through three stages: the **Conceptual model** (technology-independent, entities and relationships only), the **Logical model** (tables, attributes, keys — still DBMS-independent) and the **Physical model** (DBMS-specific storage details, ready for implementation).

Common Mistakes: Confusing the Logical model with the Physical model (Logical has no data types/indexes); assuming the Conceptual model can be implemented directly; forgetting that only the Physical model is DBMS-dependent.

2.2 Introduction to Entity Relationship Diagram (ERD)

2.2.1 Definition, Purpose and Need for ERD

Definition: An **Entity Relationship Diagram (ERD)** is a graphical/conceptual representation of the entities involved in a system and the relationships among them. It is the standard notation used to build a **conceptual data model** (introduced in Section 2.1).

Historical Background: The Entity-Relationship model was proposed by **Peter Chen in 1976** as a way to design databases by focusing on real-world "things" (entities) and how they relate, rather than on tables directly. It quickly became the standard first step of database design and is still taught and used worldwide.

Purpose of ERD

- To visually represent the data requirements of a system before implementation.
- To identify entities, their attributes, and relationships clearly and unambiguously.
- To serve as a communication tool between designers, developers and business users.
- To act as the foundation from which relational tables are later derived.

Need for ERD

- Real-world systems are complex; an ERD breaks them into manageable, visual pieces.
- Helps avoid missing requirements or misunderstanding relationships between data items.
- Provides a DBMS-independent blueprint that can later be converted into any relational schema.
- Reduces cost of redesign by catching structural errors at the design stage itself.

> **EXAM POINT**
>
> ERD is a tool used to build the **Conceptual Data Model**. It is DBMS-independent and drawn before any table is created.

Why ERD Matters in the Design Process

Consider a college wanting a new database. If developers start writing tables directly, they risk missing important relationships — for instance, forgetting that a student can enrol in multiple courses, or that a course can have multiple students. An ERD forces the designer to explicitly answer three questions for every piece of data: *What is it?* (entity), *What describes it?* (attribute), and *How is it related to other data?* (relationship). This disciplined approach is why ERDs remain the starting point of database design even decades after Chen's original 1976 paper.

> **REMEMBER**
>
> An ERD does not store data — it only models the *structure* that will later hold data. It is a design document, not the database itself.

2.2.2 Entity and Entity Set

Entity: An entity is any real-world object, person, place, event or concept about which data is stored, and which can be distinctly identified. Examples: a student, a book, an employee, an order.

Entity Type defines the common structure — the set of attributes — shared by a group of similar entities, while each individual occurrence of that structure (e.g., the specific student "Rahul", Roll No. 12) is called an **entity instance**.

Entity Set: An entity set is the full collection of entity instances belonging to the same entity type. For example, all students in a college together form the entity set *STUDENT*.

> **REMEMBER**
>
> An ER diagram shows entity *sets* (the structure/type), not individual entity instances — just as a table definition shows column names, not actual data rows.

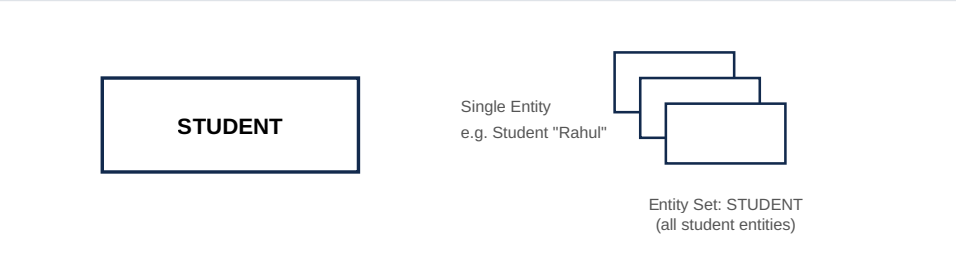


Fig. 2.2 — Entity (rectangle notation) vs Entity Set

A single rectangle represents one entity type; a stack of rectangles is often used to represent the entity set — the full collection of entity occurrences.

Table 2.2 — Entity vs Entity Type vs Entity Set

Basis	Entity	Entity Type	Entity Set
Meaning	A single, distinctly identifiable real-world object (an instance)	The common structure/attributes shared by similar entities	Collection of all instances of that entity type
Example	Student "Rahul", Roll No. 12	The definition "STUDENT (Roll_No, Name, DOB)"	All students of a college — "STUDENT"
Represents	An occurrence	A structure/template	A type / category, drawn in the ERD
Notation	Not drawn separately in an ERD	Implied by the rectangle's attributes	Rectangle labelled with the set name

2.2.3 Attributes and Types of Attributes

Attribute: An attribute is a property or characteristic of an entity that describes it, such as Name, Age, or Roll_No for the entity STUDENT.

Table 2.3 — Types of Attributes

Type	Meaning	Example	Notation
Simple (Atomic)	Cannot be divided further	Age, Roll_No	Single ellipse
Composite	Can be divided into sub-parts	Name → First_Name, Last_Name	Ellipse with child ellipses
Single-valued	Holds only one value per entity	Date_of_Birth	Single ellipse
Multi-valued	Can hold more than one value	Phone_No (a person may have several)	Double ellipse
Derived	Computed from another attribute; not stored directly	Age derived from Date_of_Birth	Dashed ellipse
Key attribute	Uniquely identifies each entity	Roll_No, Student_ID	Underlined text in ellipse

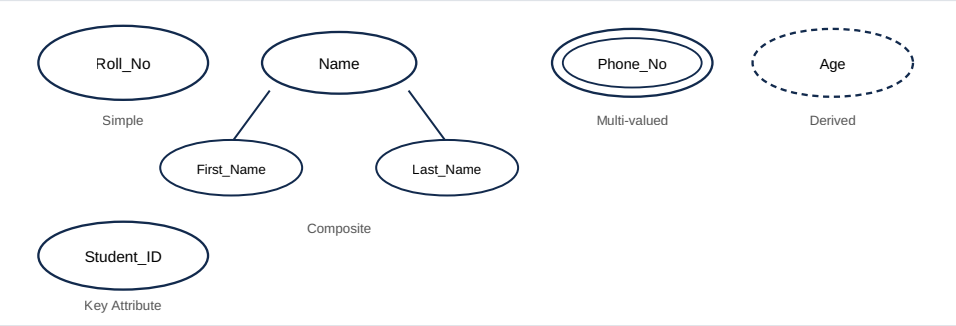


Fig. 2.3 — Notation for Simple, Composite, Multi-valued, Derived and Key attributes

Composite attributes branch into sub-attributes; multi-valued attributes use a double outline; derived attributes use a dashed outline; key attributes are underlined.

> **⚠ COMMON MISTAKE**
>
> Students often draw a derived attribute (like Age) as a normal stored attribute. Remember: if a value can be calculated from another attribute, it should be shown as *derived* (dashed ellipse), not stored separately.

2.2.4 Relationship and Degree of Relationship

Relationship: A relationship is an association between two or more entities. For example, a STUDENT "enrolls in" a COURSE. In ER notation, a relationship is shown as a **diamond** connecting the related entities.

Just as "entity" and "entity set" are distinguished, ER modelling separates a **Relationship Type** from a **Relationship Set**:

- **Relationship Type:** The association itself, at the level of entity sets — e.g., the type "Enrolls in" connecting STUDENT and COURSE. This is what is actually drawn (as a diamond) in the ER diagram.
- **Relationship Set:** The collection of all actual instances of that relationship — e.g., every real enrolment record such as "Student S1 enrolled in Course C2", "Student S2 enrolled in Course C1", and so on.

> **REMEMBER**
>
> Relationship Type is to Relationship Set what Entity Type is to Entity Set — one is the design-time definition, the other is the run-time collection of actual links.

Degree of Relationship: The degree indicates the number of entity sets participating in a relationship.

Table 2.4 — Degree of Relationship

Degree	Name	Meaning	Example
1	Unary (Recursive)	An entity relates to another entity of the same set	An EMPLOYEE "supervises" another EMPLOYEE
2	Binary	Two different entity sets participate	STUDENT "enrolls in" COURSE
3	Ternary	Three entity sets participate together	SUPPLIER "supplies" PART to PROJECT
n	N-ary	General case — n entity sets participate together	Ternary is simply the N-ary case where n = 3

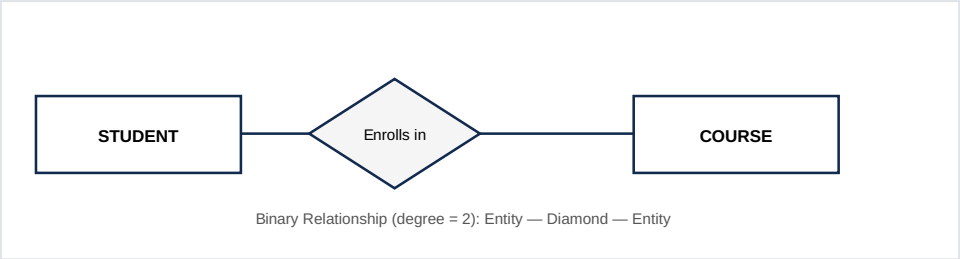


Fig. 2.4 — Relationship notation (rectangle = entity, diamond = relationship)

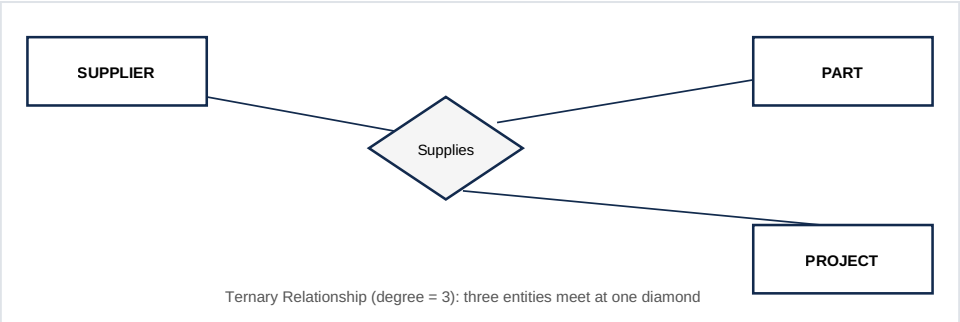


Fig. 2.5 — Ternary relationship example: SUPPLIER supplies PART to PROJECT

Unlike a binary relationship, a ternary relationship cannot always be broken into two separate binary relationships without losing meaning — all three entities must be considered together.

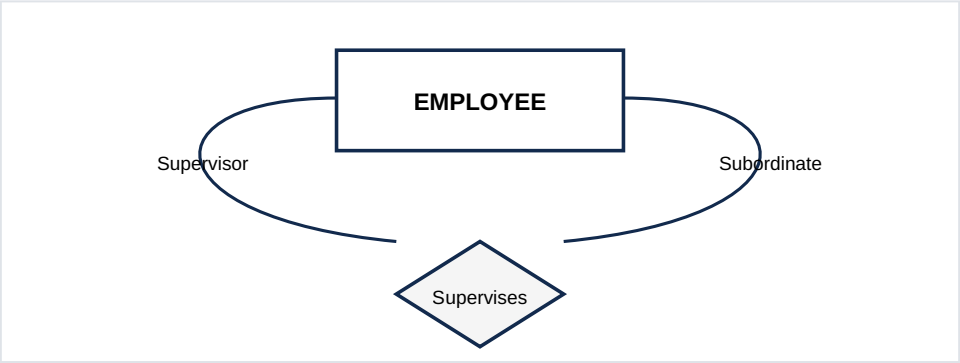


Fig. 2.5(a) — Unary (recursive) relationship: EMPLOYEE supervises EMPLOYEE

Both connecting lines return to the same entity set, but each is labelled with the role it plays (Supervisor / Subordinate) so the two participations are not confused.

2.2.5 Cardinality and Participation Constraints

Cardinality defines the maximum number of times an entity in one entity set can be associated with entities in another set through a relationship.

Table 2.5 — Types of Cardinality

Type	Meaning	Example
One-to-One (1:1)	One entity of A relates to exactly one entity of B	One PERSON has one PASSPORT
One-to-Many (1:M)	One entity of A relates to many entities of B	One DEPARTMENT has many EMPLOYEES
Many-to-One (M:1)	Many entities of A relate to one entity of B	Many STUDENTS belong to one CLASS
Many-to-Many (M:N)	Many entities of A relate to many entities of B	Many STUDENTS enroll in many COURSES

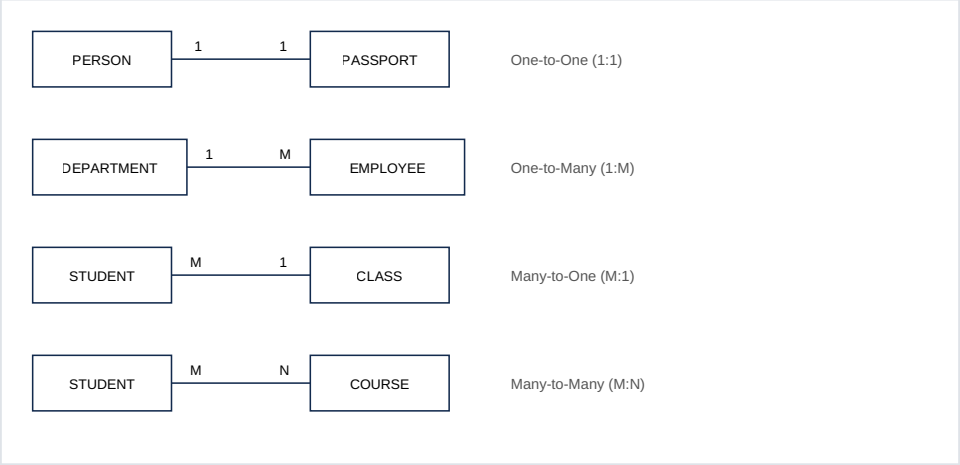


Fig. 2.6 — The four cardinality types with real-world examples

Cardinality can also be pictured using **sets** — mapping each element of one entity set to element(s) of another. This view makes Many-to-Many relationships especially easy to verify.

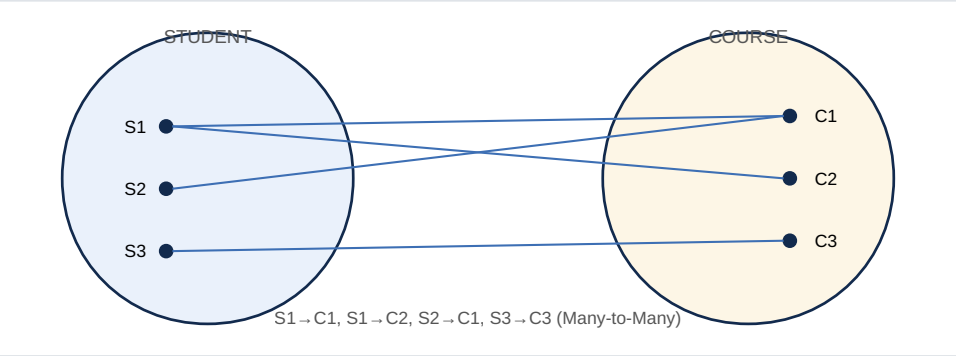


Fig. 2.6(a) — Set representation of a Many-to-Many relationship

Each dot in STUDENT can connect to one or more dots in COURSE, and each COURSE dot can likewise be reached from more than one STUDENT — confirming the relationship is genuinely M:N.

Participation Constraints

- **Total Participation:** Every entity in the set must participate in the relationship (shown with a double line). Example: every EMPLOYEE must belong to some DEPARTMENT.
- **Partial Participation:** Only some entities in the set need to participate (single line). Example: not every EMPLOYEE manages a project.

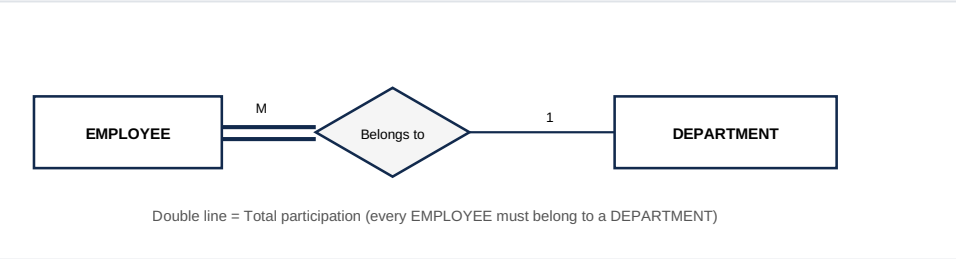


Fig. 2.6(b) — Total participation shown using a double connecting line

> **REMEMBER**
>
> Cardinality tells "how many"; participation tells "whether compulsory or optional".

2.2.6 Keys used in ERD

Table 2.6 — Primary Key vs Candidate Key vs Composite Key vs Foreign Key

Key	Definition	Example
Candidate Key	An attribute (or set of attributes) that can uniquely identify an entity; there may be more than one candidate key	Roll_No, Email in STUDENT
Primary Key	The candidate key chosen to uniquely identify each entity in the entity set	Roll_No selected as the Primary Key
Composite Key	A primary key made up of two or more attributes combined	(Order_ID + Product_ID) in an ORDER_DETAIL entity
Foreign Key	An attribute in one entity that refers to the primary key of another entity, used to establish a relationship	Dept_ID in EMPLOYEE referring to DEPARTMENT

EXAM POINT

Every Primary Key is a Candidate Key, but not every Candidate Key is chosen as the Primary Key.

2.2.7 Naming Conventions and Steps to Create an ER Diagram

Naming Conventions

- Entity names are usually written in singular, uppercase nouns (e.g., STUDENT, not Students).
- Attribute names are written in lowercase or mixed case, clearly describing the property (e.g., Roll_No, Date_of_Birth).
- Relationship names are usually verbs or verb phrases describing the action (e.g., Enrolls, Supervises, Places).

Steps to Create an ER Diagram

1. **Identify entities** — determine the main objects about which data must be stored.
2. **Identify relationships** — determine how the entities are associated with each other.
3. **Identify attributes** — list the properties of each entity and each relationship (if any).
4. **Identify key attributes** — decide the candidate keys and select a primary key for each entity.
5. **Determine cardinality and participation** — decide 1:1, 1:M, M:1, M:N and whether participation is total or partial.
6. **Draw the diagram** — use rectangles (entities), ellipses (attributes) and diamonds (relationships), connected with lines.
7. **Review and refine** — validate the diagram against requirements and remove redundancy or ambiguity.

> **SHORTCUT**
>
> Remember the steps as **"E-R-A-K-C-D-R"**: Entities → Relationships → Attributes → Keys → Cardinality → Draw → Review.

2.2.8 Advantages, Limitations and Applications of ERD

Like any modelling tool, the ER diagram has clear strengths that make it the standard first step in database design, as well as boundaries beyond which other techniques (such as normalisation or detailed process modelling) must take over.

Advantages

- Simple, visual and easy to understand for both technical and non-technical users.
- Provides a clear DBMS-independent blueprint of the database.
- Helps in identifying redundant or missing data requirements early.
- Serves as documentation that can be referred to throughout the system's life.
- Forms the direct basis for creating relational tables.

Limitations

- Cannot represent complex business rules or procedural logic.
- Becomes difficult to read when the number of entities is very large.
- Does not show how data will actually be processed or manipulated.
- Different designers may model the same requirement differently, causing inconsistency.

Applications

- Designing databases for academic systems (Student, College Management).
- Designing databases for Library, Hospital and Banking systems (see worked examples below).
- Requirement analysis and documentation in software engineering projects.
- Communicating database structure to stakeholders during system design reviews.

2.2.9 Common Errors while Drawing ERDs

- Treating an attribute as an entity, or an entity as an attribute (e.g., making "Address" a separate entity when it is simply an attribute of STUDENT).
- Forgetting to mark the primary key attribute (should be underlined).
- Showing a derived attribute as a normal stored attribute.
- Incorrectly specifying cardinality (e.g., writing 1:1 where the real relationship is 1:M).
- Missing participation constraints (total vs partial) where they matter.
- Using vague or inconsistent naming for entities, attributes and relationships.
- Creating redundant relationships that repeat information already captured elsewhere.

> **⚠ COMMON MISTAKE**
>
> A very common exam trap: students confuse a *weak entity* (which depends on another entity for its identity, shown with a double rectangle) with a normal (strong) entity.

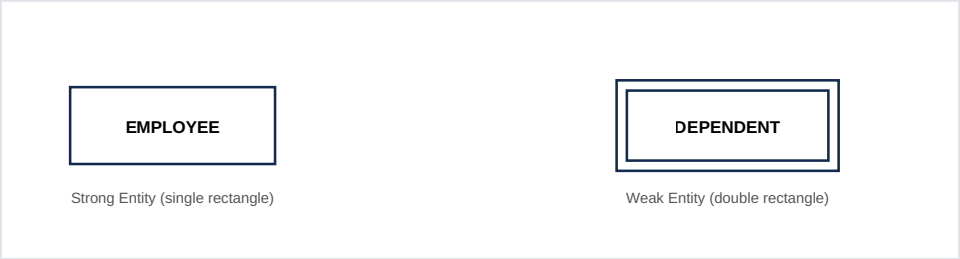


Fig. 2.7 — Strong Entity vs Weak Entity notation

A weak entity (e.g., DEPENDENT of an EMPLOYEE) cannot be uniquely identified by its own attributes alone and depends on a related strong entity for identification.

When a weak entity is connected to its owning strong entity, the connecting relationship is called an **identifying relationship** and is drawn as a **double diamond**. The weak entity's partial key (an attribute that only distinguishes it *within* its owner, not globally) is shown with a *dashed underline* instead of a solid one.

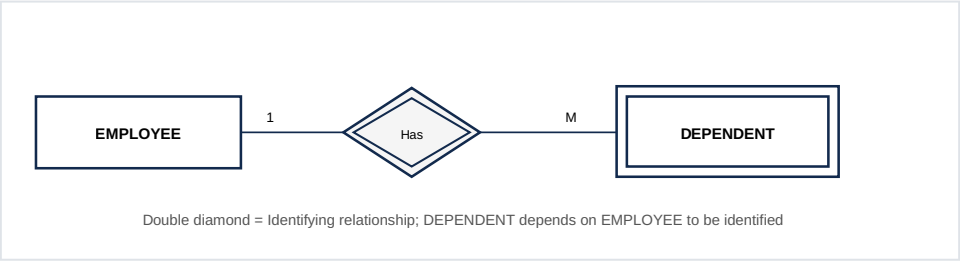


Fig. 2.7(a) — Weak entity connected to its owner via an identifying relationship

2.2.10 Worked Examples

The following examples show how the full ERD process — identifying entities, attributes, keys and cardinality — is applied to common real-world systems. Before working through them, Fig. 2.11(a) below summarises every notation symbol introduced so far, for quick reference while reading the diagrams.

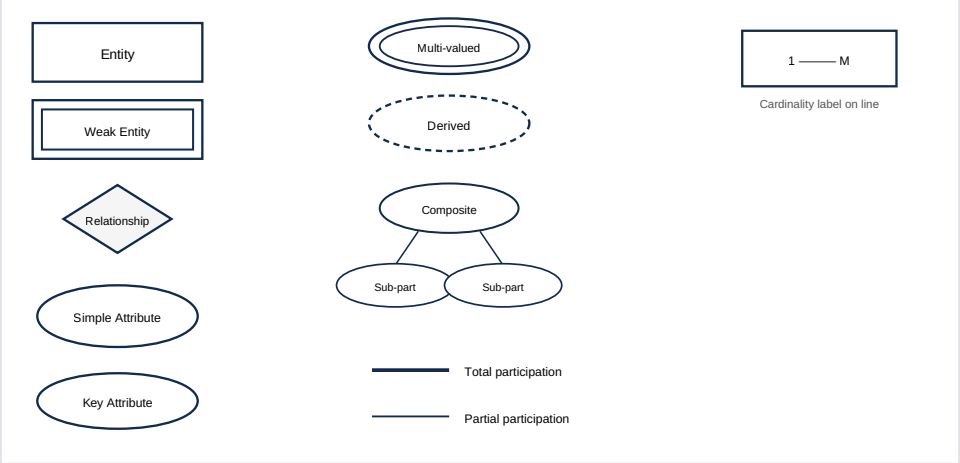


Fig. 2.11(a) — Quick-reference chart: all ER notation symbols used in this chapter

Example 1: Student–Course Database

Step 1 (Entities): STUDENT, COURSE.
Step 2 (Relationship): STUDENT "Enrolls in" COURSE.
Step 3 (Attributes): STUDENT → Roll_No (key), Name, DOB; COURSE → Course_ID (key), Title, Credits.
Step 4 (Cardinality): Many-to-Many — a student enrolls in many courses, and a course has many students.

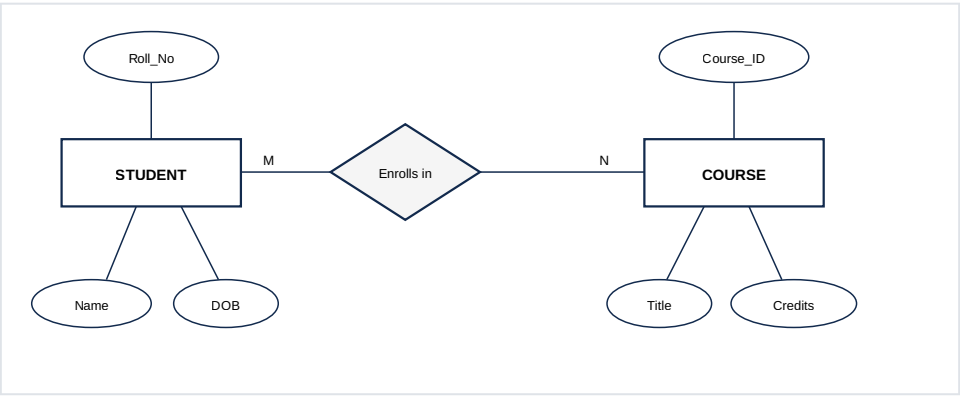


Fig. 2.8 — ER Diagram: Student–Course System (M:N relationship)

Example 2: College Database

Entities: STUDENT, DEPARTMENT, TEACHER, COURSE. **Relationships:** DEPARTMENT "Offers" COURSE (1:M); TEACHER "Teaches" COURSE (1:M); STUDENT "Belongs to" DEPARTMENT (M:1). **Key attributes:** Student_ID, Dept_ID, Teacher_ID, Course_ID. This example extends Example 1 by adding the organisational entities that typically surround a Student–Course core in a real college system.

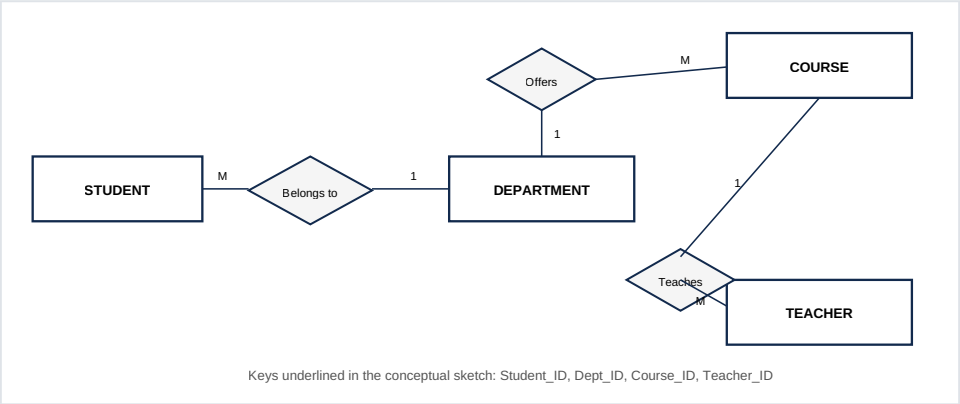


Fig. 2.8(a) — ER Diagram: College Database (Student, Department, Course, Teacher)

A realistic college system combines several binary relationships around a common core; each relationship is modelled and labelled independently with its own cardinality.

Example 3: Library Management System

Entities: MEMBER, BOOK. **Relationship:** MEMBER "Borrows" BOOK. **Attributes:** MEMBER → Member_ID (key), Name, Address; BOOK → Book_ID (key), Title, Author. **Cardinality:** Many-to-Many (a member borrows many books over time; a book can be borrowed by many members over time).

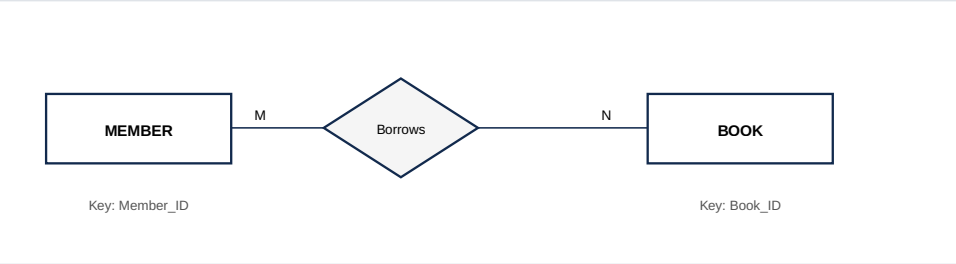


Fig. 2.9 — ER Diagram: Library Management System

Example 4: Hospital Management System

Entities: PATIENT, DOCTOR. **Relationship:** DOCTOR "Treats" PATIENT. **Attributes:** PATIENT → Patient_ID (key), Name, Age; DOCTOR → Doctor_ID (key), Name, Specialisation. **Cardinality:** One-to-Many — one doctor treats many patients (a simplified view; in reality, this can also be modelled as M:N if a patient sees multiple doctors).

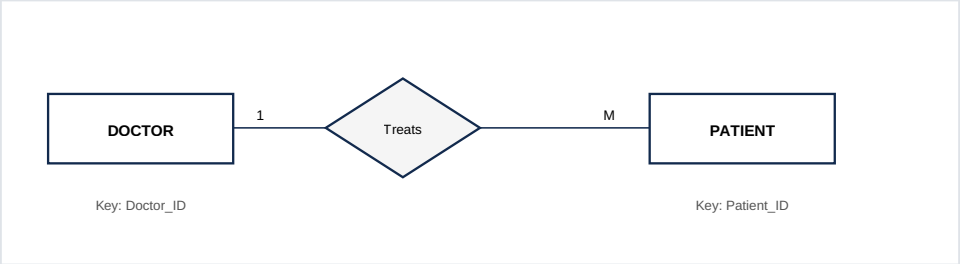


Fig. 2.10 — ER Diagram: Hospital Management System

Example 5: Banking System

Entities: CUSTOMER, ACCOUNT. **Relationship:** CUSTOMER "Holds" ACCOUNT. **Attributes:** CUSTOMER → Customer_ID (key), Name, Address; ACCOUNT → Account_No (key), Type, Balance (derived/updatable). **Cardinality:** One-to-Many — one customer may hold many accounts (savings, current); each account, in this simplified model, belongs to one customer.

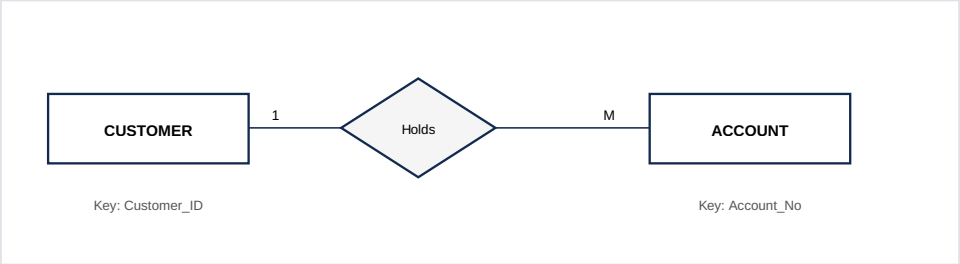


Fig. 2.11 — ER Diagram: Banking System

> **EXAM POINT**
>
> In worked examples, always show the process: Entities → Relationship → Attributes (with keys underlined) → Cardinality. Examiners award marks step-by-step, not only for the final diagram.

> **EXAM TIP — Section 2.2**
>
> When asked to "design an ERD" for a scenario in an exam, write the entities and their key attributes first in words, then draw the diagram — this way you get partial credit even if you run out of time to finish the drawing.

Chapter Summary — Section 2.2

An ERD is a graphical technique (proposed by Peter Chen, 1976) used to build the conceptual data model. It uses rectangles for entities, ellipses for attributes and diamonds for relationships. Attributes may be simple, composite, single/multi-valued, derived or key attributes. Relationships have a degree (unary, binary, ternary) and a cardinality (1:1, 1:M, M:1, M:N), along with participation constraints (total/partial). Keys — candidate, primary, composite and foreign — establish uniqueness and links between entities. A systematic seven-step process (identify entities → relationships → attributes → keys → cardinality → draw → review) is used to build correct, exam-ready ER diagrams.

Revision Section

One-Page Quick Revision Notes

Key Definitions

- **Data Model:** Blueprint of structure, relationships & constraints of data.
- **Conceptual Model:** High-level, DBMS-independent view (What data).
- **Logical Model:** Tables, attributes, keys — still DBMS-independent (How organised).
- **Physical Model:** DBMS-specific storage details (How stored).
- **ERD:** Graphical technique to model entities and relationships (Peter Chen, 1976).
- **Entity:** A distinctly identifiable real-world object.
- **Entity Set:** Collection of entities of the same type.
- **Attribute:** A property describing an entity.
- **Relationship:** Association between two or more entities.
- **Cardinality:** Maximum number of associations allowed (1:1, 1:M, M:1, M:N).

Important Differences

- Conceptual vs Logical: "What" vs "How organised".
- Logical vs Physical: DBMS-independent vs DBMS-dependent.
- Entity vs Attribute: Object vs property of the object.
- Candidate Key vs Primary Key: Possible unique identifiers vs the one chosen.
- Total vs Partial Participation: Compulsory vs optional participation.
- Strong vs Weak Entity: Independent identity vs depends on another entity.

Notation Quick Reference

- Rectangle = Entity
- Ellipse = Attribute
- Diamond = Relationship
- Double Ellipse = Multi-valued Attribute
- Dashed Ellipse = Derived Attribute
- Underline = Key Attribute
- Double Rectangle = Weak Entity

Order to Remember

- Model design order: Conceptual → Logical → Physical
- ERD steps: Entities → Relationships → Attributes → Keys → Cardinality → Draw → Review

32 One-Line Revision Points

1. A data model describes structure, relationships and constraints of data.
2. Good data models are simple, complete, non-redundant, accurate and flexible.
3. Conceptual model is technology-independent and focuses on entities/relationships.
4. Logical model defines tables, attributes and keys but is still DBMS-independent.
5. Physical model is DBMS-specific and ready for actual implementation.
6. Design order is always Conceptual → Logical → Physical.
7. ERD was proposed by Peter Chen in 1976.
8. ERD is used to build the conceptual data model.
9. Entity = a real-world object; Entity Set = collection of similar entities.
10. Attribute = a property describing an entity.
11. Simple attributes cannot be divided further; composite attributes can.
12. Single-valued attributes hold one value; multi-valued attributes hold many.
13. Derived attributes are computed, not stored (e.g., Age from DOB).
14. Key attributes uniquely identify an entity and are shown underlined.
15. Relationship = association between two or more entities, shown as a diamond.
16. Relationship Type is the design-time diamond; Relationship Set is the actual collection of linked instances.
17. Entity Type defines the structure; Entity Set is the collection of instances of that type.
18. Degree of relationship = number of participating entity sets.
19. Unary relationship involves one entity set; binary involves two; ternary involves three.
20. Cardinality defines the maximum association count between entities.
21. 1:1, 1:M, M:1 and M:N are the four types of cardinality.
22. Total participation means every entity must take part in the relationship.
23. Partial participation means participation is optional.
24. Candidate key: any attribute that could uniquely identify an entity.
25. Primary key: the candidate key actually chosen for identification.
26. Composite key: a primary key formed of two or more attributes together.
27. Foreign key: an attribute referring to the primary key of another entity.
28. Weak entities depend on a related (strong) entity for identification.
29. Weak entities are shown using a double rectangle.
30. ERDs are simple and visual, but cannot represent procedural business logic.
31. Common ERD mistake: confusing an attribute with an entity.
32. ERD forms the direct basis for deriving relational database tables.

Expected Viva Questions

These questions test whether a candidate truly understands the concepts, not just the definitions — common in interview-style or practical viva rounds for TGT, MCA and BCA examinations.

- Why is the Conceptual model considered technology-independent?
- Give one real-life example each of a derived and a multi-valued attribute.
- Why can a relationship itself sometimes have attributes?
- How would you convert a Many-to-Many relationship into relational tables?
- What happens if a primary key is not defined for an entity?
- Why must a weak entity always be linked to a strong (owner) entity?
- If two attributes could both serve as a primary key, what would you call each of them before one is chosen?
- Why is it usually wrong to model "Address" as a separate entity for a STUDENT?
- In the Banking example, why is Balance often treated as a derived/updatable attribute rather than a fixed stored value?
- What is lost if a ternary relationship is incorrectly split into two binary relationships?
- Why does the Physical model change if the target DBMS is changed, while the Logical model does not?

FINAL EXAM TIP

Whenever a question gives you a real-world scenario, follow the seven-step ERD method in order — Entities, Relationships, Attributes, Keys, Cardinality, Draw, Review — and label your diagram clearly. Partial marks are commonly awarded for each correct step even if the final diagram is incomplete.